Towards Practical On-Device Android Malware Detection via Lightweight Static Analysis

Abdullah Faiyaz
Department of CSE, BUET
2005065@ugrad.cse.buet.ac.bd

Jarin Tasnim
Department of CSE, BUET
2005083@ugrad.cse.buet.ac.bd

Mohammad Sadnam Faiyaz
Department of CSE, BUET
2005111@ugrad.cse.buet.ac.bd

Md. Mostofa Akbar
Department of CSE, BUET
mostofa@cse.buet.ac.bd

Abstract—Android malware keeps growing, yet accurate detectors often depend on cloud backends or heavyweight analysis ill-suited to private, offline screening on commodity phones. We present **VigiDroid**, an on-device static pre-screener that orchestrates literature-derived lightweight models through ONNX Runtime without uploading APK data. Detectors are trained offline on a large temporal APK corpus, exported with train/serve parity checks, and deployed as verified bundles together with a calibrated cascade policy. On the phone, VigiDroid parses each APK once, runs a four-tier early-exit cascade, and logs per-stage resource metrics as append-only JSONL for thesis evaluation.

Index Terms— Android malware detection, static analysis, on-device inference, ONNX, ensemble learning, mobile security.

I. Introduction

Static malware analysis reports strong offline accuracy, yet few works optimize deployment on phones with limited CPU, memory, and battery. **VigiDroid** re-implements ten lightweight static detectors in ONNX and profiles accuracy alongside extraction cost on real hardware. **Dex+Broadcast fusion** reaches 97.0% accuracy and F1 of 0.950 at 0.5 ms mean stage time.

II. Background and Related Works

Android malware detection spans permission-based linear models [2], MLDP feature selection [7], broadcast-receiver mining [3], byte-level 1-D CNNs on APK tails [1], and multi-branch DEX/manifest fusion inspired by MSF-Droid [8]. SeqMobile [4] showed on-device RNN screening over permissions and API traces; SAMADroid [5] combined lightweight local checks with cloud dynamic analysis, but modern Android privilege restrictions limit that pattern. DL-Droid [6] executed apps on real hardware for dynamic labels, which is accurate yet impractical for routine offline screening. Overall, prior work achieves strong offline accuracy but rarely reports deployment-aware trade-offs on commodity phones.

III. Proposed Methodology

PC model development (Fig. 1). Each detector follows an eight-phase pipeline (P0–P8): index a ∼500 GB temporal APK corpus, extract features, train on 2020–2021 samples, evaluate on 2022–2023 holdout, export ONNX bundles with thresholds and vocabularies, and pass a parity gate ($\Delta p \leq 10^{-4}$) before any Android integration. **On-device system (Fig. 2).** VigiDroid runs inference locally via ONNX Runtime, parses each APK once into a shared `FeatureContext`, executes a calibrated four-tier cascade with early exit, fuses tier scores into `benign`/`malware`/`uncertain` verdicts, and appends per-stage wall time, CPU, memory, and battery metrics to JSONL for thesis analysis.

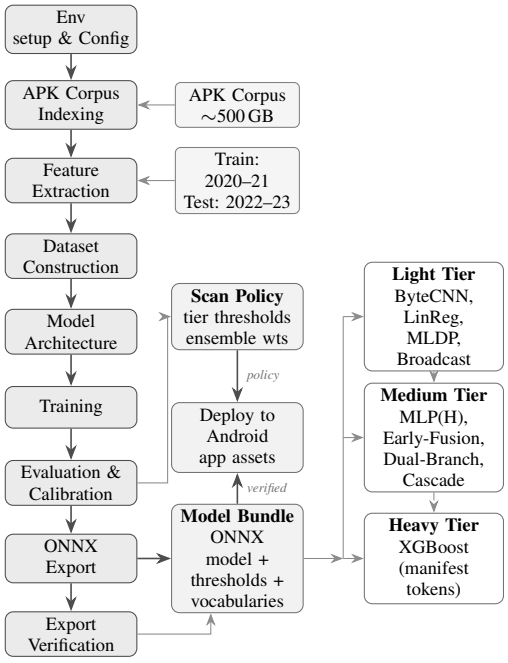

Fig. 1: Offline model development pipeline

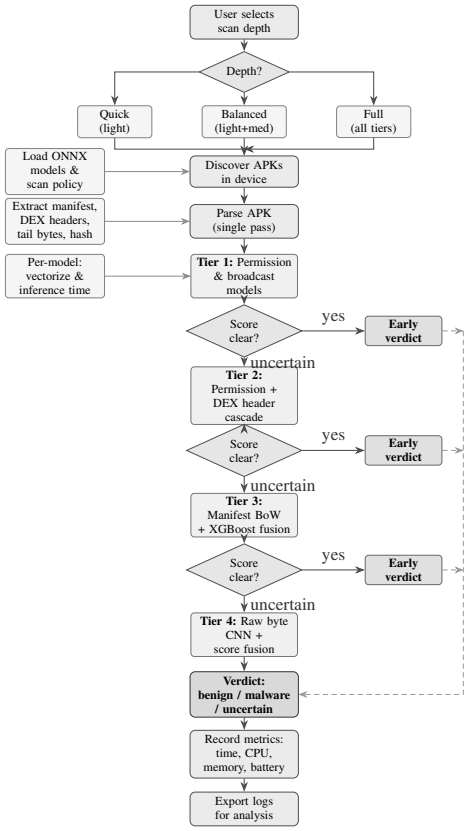

Fig. 2: VigiDroid on-device four-tier cascade with early exit

TABLE I: Model comparison: offline test metrics and on-device stage averages (from device JSONL). Memory in MB; battery in %.

Method	Acc.	F1	AUC	CPU	Mem.	Batt.	Time	Feas.	Comments
XGBoost	0.937	0.888	0.991	97.4	0.02	0.48%	100 ms	low	Strong AUC; heavy cost
1D-CNN	0.901	0.849	0.959	1.7	0.01	0.01%	1.7 ms	low	Byte-tail; tier-4
MLP(H)	0.969	0.948	0.968	0.7	0.01	<0.01%	0.4 ms	high	Fast, accurate
Early-Fusion	0.941	0.906	0.969	7.8	0.03	0.04%	7.9 ms	medium	Tier-3 fused
Dual-Branch	0.969	0.948	0.981	7.9	0.02	0.04%	7.9 ms	medium	Ablation only
LinRegDroid	0.918	0.865	0.926	1.0	0.00	<0.01%	0.7 ms	low	Linear baseline
MLDP-pruned	0.932	0.886	0.902	0.6	0.00	<0.01%	0.4 ms	medium	Tier-1 gate
Broadcast+MLDP	0.936	0.893	0.937	1.1	0.00	<0.01%	0.4 ms	medium	Receiver hybrid
MLDP+DEX casc.	0.958	0.930	0.947	0.8	0.01	<0.01%	0.4 ms	high	Tier-2 fuse
Dex+Broadcast	0.970	0.950	0.979	0.9	0.01	<0.01%	0.5 ms	high	Peak accuracy
DroidCat	0.974	0.978	0.980	–	–	–	–	–	–
SeqMobile	0.977	0.977	–	–	–	–	–	low	–

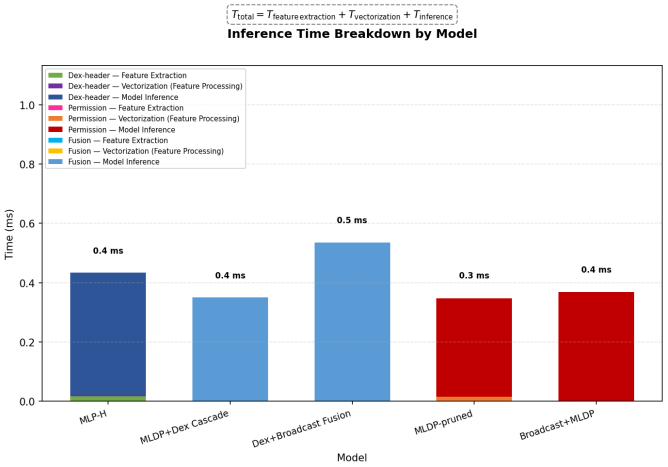

Fig. 3: Inference time vs. APK size across models.

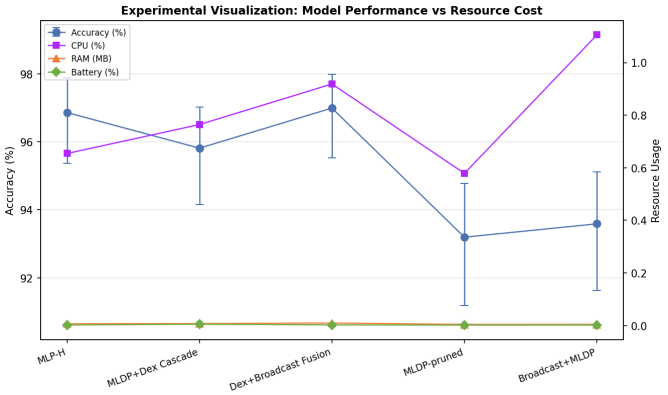

Fig. 4: Accuracy vs. on-device resource cost (sample models).

IV. Experiments

Table I compares offline quality against on-device feasibility for all ten thesis models. **Dex+Broadcast** attains the highest accuracy (97.0%, F1 0.950) at 0.5 ms mean stage time, while **MLP(H)** and **MLDP+DEX cascade** offer comparable F1 with sub-millisecond latency and high feasibility. **XGBoost** remains strongest on AUC but costs ∼100 ms per stage because of multi-DEX manifest parsing. Fig. 3 shows extraction and inference scaling with APK size; Fig. 4 highlights the accuracy–resource Pareto frontier, motivating tiered early exit over always-on heavy models. **Protocol.** Labeled eval APKs (`scan_XXXX_benign/malware.apk`, 1,528 samples from the 2022–2023 holdout) are pushed via ADB to `/sdcard/Download/Scanable`, scanned on-device, and JSONL metrics are pulled post-run; offline labels follow the temporal corpus year splits.

V. Conclusion

VigiDroid unifies literature-based static detectors through a train-export-deploy pipeline and four-tier ONNX cascade, logging resource metrics alongside verdicts for deployment-aware model selection.

References

[1] C. Hasegawa and H. Iyatomi, "One-dimensional convolutional neural networks for Android malware detection," in *Proc. ICCE*, 2020.

[2] A. Narayanan et al., "LinRegDroid: Detection of Android malware using multiple linear regression," *Computers & Security*, 2018.

[3] M. Mohsen and A. Ismail, "Detecting Android malwares by mining statically registered broadcast receivers," *Int. J. Advanced Computer Science and Applications*, 2017.

[4] Ruitao Feng et al., "SeqMobile: An Efficient Sequence-Based Malware Detection System Using RNN on Mobile Devices" *25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, 2020.

[5] Saba Arshad et al., "SAMADroid: A Novel 3-Level Hybrid Malware Detection Model for Android Operating System" *IEEE Access*, 2018.

[6] Mohammed K. Alzaylaee et al., "DL-Droid: Deep learning based android malware detection using real devices" *IEEE Access*, 2020.

[7] S. Wang et al., "Permission extraction framework for Android malware detection," *IEEE Access*, 2019.

[8] Y. Li et al., "MSFDroid: Multi-stage fusion for Android malware detection," *IEEE Trans. Information Forensics and Security*, 2022.

[9] D. Arp et al., "Drebin: Effective and explainable detection of Android malware in your pocket," in *Proc. NDSS*, 2014.